CH
R

Clinical Health Records System — CHR-System

Product Requirements Document · v2.0 · Full Redesign & Evolution Blueprint

Next.js 15 | TypeScript | PostgreSQL | Prisma ORM | Open Source

AUTHOR	REPOSITORY	VERSION	DATE
Madhavan Shivakumar	Madhavan-dev18/chr-system	2.0 — Full Redesign	June 2026

This document defines the complete architecture, feature roadmap, security design, database schema, API specification, CI/CD strategy, and UI/UX vision for transforming chr-system into a production-grade, open-source Clinical Health Records platform suitable for real healthcare organizations and as a flagship portfolio project.

Table of Contents

01. Executive Summary

CHR-System is a Next.js 15 health-records application with 53 commits, representing significant real engineering effort. However, it currently ships under a Firebase Studio boilerplate README that completely hides its depth. This PRD transforms chr-system from an undocumented demo into a fully production-ready, HIPAA-aligned, open-source Electronic Health Records (EHR) platform — the most complete free EHR on GitHub.

53	0	100%	OWASP Top 10	>80%
Commits Already In	Recurring Cost	Open Source	Security Coverage	Target Test Coverage

Strategic Objectives

- Transform into the #1 free, open-source EHR platform on GitHub
- Production-grade security: JWT + RBAC + audit logs + OWASP Top 10 hardening
- Neumorphic UI system (as per reference design) — soft, modern, accessible
- AI-powered clinical decision support using free/open-source models (Ollama, HuggingFace)
- One-command Docker setup — zero manual configuration
- Comprehensive CI/CD with GitHub Actions (free tier only)
- HIPAA-aligned audit trail — every record access logged with who/when/what
- >80% automated test coverage (unit + integration + E2E + security)

02. Repository Audit — Current State

What Exists (Verified)

Despite the misleading boilerplate README, chr-system contains real, functional Next.js 15 / TypeScript code committed over 53 meaningful commits. The following was confirmed through actual code inspection:

Asset	Tech / Detail	Status
Framework	Next.js 15 (App Router)	✓ Confirmed
Language	TypeScript throughout	✓ Confirmed
Auth Provider	Firebase Auth (current)	■ Replace
Database	Firebase Firestore (current)	■ Replace
UI Library	shadcn/ui components	✓ Keep
Commit Count	53 commits — real work	✓ Confirmed
README	Firebase Studio boilerplate	✗ Critical Gap
Tests	None / empty stubs	✗ Critical Gap
CI/CD	None configured	✗ Critical Gap
Docker	Not present	✗ Gap
RBAC	Not implemented	✗ Critical Gap
Audit Log	Not implemented	✗ Critical Gap

Critical Findings

Severity	Finding
CRITICAL	README shows Firebase Studio boilerplate — 53 commits of real work completely hidden from recruiters
CRITICAL	Zero test coverage — no unit, integration, or E2E tests present
CRITICAL	No authentication role separation (admin vs doctor vs patient vs nurse)
HIGH	Firebase lock-in — vendor dependency with free-tier limits and no SQL querying
HIGH	No CI/CD pipeline — no automated quality gates on push/PR
HIGH	No audit logging — HIPAA/healthcare requires every record access to be logged
MEDIUM	No Docker setup — onboarding requires manual Firebase project configuration
MEDIUM	No API documentation — endpoints undiscoverable for other developers
LOW	No performance monitoring or error tracking integrated

03. Architecture Redesign

From Firebase → Production Stack

The redesign replaces Firebase with a fully open-source, self-hostable stack that eliminates vendor lock-in, provides SQL query power, and enables HIPAA-grade audit trails.

Layer	Current	Target	Reason
Frontend	Next.js 15 + shadcn	Next.js 15 + shadcn + Tailwind	**Keep — already strong**
Auth	Firebase Auth	NextAuth v5 + bcrypt + JWT	RBAC, no vendor lock-in
Database	Firebase Firestore	PostgreSQL 16 + Prisma ORM	SQL, relations, ACID
ORM	None (raw Firestore)	Prisma 5 with migrations	Type-safe, auditable
Cache	None	Redis 7 (Upstash free / Docker)	Rate limiting, sessions
File Store	Firebase Storage	MinIO (self-hosted S3-compatible)	Zero cost, open-source
AI/ML	None	Ollama + HuggingFace Inference	Free, local, privacy-safe
Search	None	Meilisearch (Docker)	Fast patient/record search
Background Jobs	None	BullMQ + Redis	PDF generation, notifications
Monitoring	None	OpenTelemetry + Prometheus + Grafana	Free, self-hosted
Error Tracking	None	Sentry (free OSS tier)	Real-time error capture
API Docs	None	Swagger UI + tRPC	Auto-generated, typed
Email	None	Nodemailer + SMTP (free)	Appointment reminders
Container	None	Docker + Docker Compose	One-command setup

Folder Structure

Clean, role-separated monorepo layout:

```
chr-system/
■■■ apps/
■   ■■■ web/                    ← Next.js 15 App Router
■       ■■■ app/
■       ■   ■■■ (auth)/         ← Login, Register, Forgot Password
■       ■   ■■■ (dashboard)/
■       ■   ■   ■■■ admin/      ← System admin panel
■       ■   ■   ■■■ doctor/     ← Doctor workspace
■       ■   ■   ■■■ nurse/      ← Nursing station
■       ■   ■   ■■■ patient/    ← Patient portal
■       ■   ■■■ api/            ← Next.js API Routes (REST + tRPC)
■       ■■■ components/
■       ■   ■■■ ui/             ← shadcn base + neumorphic overrides
■       ■   ■■■ forms/          ← Zod-validated form components
```

```
■       ■   ■■■ charts/        ← Recharts health data visualizations
■       ■■■ lib/
■           ■■■ auth.ts        ← NextAuth config
■           ■■■ prisma.ts      ← Prisma singleton
■           ■■■ audit.ts       ← Audit log utility
■■■ packages/
■   ■■■ db/                    ← Prisma schema + migrations
■   ■■■ types/                 ← Shared TypeScript types
■   ■■■ validators/            ← Zod schemas (shared)
■■■ docker/
■   ■■■ docker-compose.yml     ← Full stack (pg, redis, minio, meilisearch)
■   ■■■ docker-compose.dev.yml ← Dev overrides
■■■ .github/workflows/         ← CI/CD pipelines
■■■ docs/                      ← ADRs, API docs, deployment guides
```

04. Database Schema Redesign

PostgreSQL 16 with Prisma ORM. Schema designed for HIPAA audit compliance, multi-tenancy (clinic/hospital), and efficient clinical querying.

Table	Key Fields	Purpose
users	id, email, password_hash, role (ENUM), clinic_id, is_active	All system users with RBAC
clinics	id, name, license_no, address, timezone	Multi-clinic support
patients	id, mrn, name, dob, blood_type, allergies[], clinic_id	Patient demographics
doctors	id, user_id, specialization, license_no, availability (JSON)	Doctor profiles
appointments	id, patient_id, doctor_id, scheduled_at, status (ENUM), notes	Scheduling engine
medical_records	id, patient_id, doctor_id, type, encrypted_content, attachments[]	EHR core
prescriptions	id, record_id, medication, dosage, frequency, start_date, end_date	Medication management
vitals	id, patient_id, recorded_at, bp_systolic, bp_diastolic, heart_rate, spo2	Time-series vitals
lab_results	id, patient_id, test_name, result_value, unit, reference_range, status	Lab integration
audit_logs	id, user_id, action, resource, resource_id, ip, user_agent, timestamp	HIPAA audit trail
notifications	id, user_id, type, channel (SMS/EMAIL/PUSH), payload, sent_at	Notification center
billing	id, patient_id, appointment_id, amount, status, insurance_claim_id	Basic billing module

Critical Schema Decisions

- medical_records.encrypted_content — AES-256-GCM encryption at application layer, not just DB
- audit_logs — append-only table with NO UPDATE/DELETE grants for the app DB user
- Role ENUM: ADMIN | DOCTOR | NURSE | PATIENT | RECEPTIONIST | LAB_TECH
- Soft deletes everywhere (deleted_at timestamp) — never hard delete medical data
- clinic_id foreign key on all clinical tables — multi-tenancy at row level
- Prisma RLS (Row Level Security) middleware to enforce role-based data access
- UUID v7 primary keys — time-sortable, globally unique, no sequential guessing

05. API Redesign

RESTful + tRPC Hybrid Architecture

Public-facing endpoints use REST (for third-party integration). Internal Next.js client-server calls use tRPC for end-to-end type safety.

Method	Endpoint	Auth	Description
POST	/api/auth/login	Public	Email+password → JWT access + refresh tokens
POST	/api/auth/refresh	Refresh JWT	Rotate access token
POST	/api/auth/logout	JWT	Invalidate refresh token in Redis
GET	/api/patients	DOCTOR/NURSE/ADMIN	List patients (paginated, searchable)
POST	/api/patients	ADMIN/RECEPTIONIST	Register new patient
GET	/api/patients/:id	DOCTOR/NURSE	Get full patient profile
GET	/api/patients/:id/records	DOCTOR	Medical records (audit logged)
POST	/api/patients/:id/records	DOCTOR	Create new medical record
GET	/api/patients/:id/vitals	DOCTOR/NURSE	Vitals time series
POST	/api/patients/:id/vitals	NURSE	Record new vitals
GET	/api/appointments	Role-scoped	List appointments (doctor/patient view)
POST	/api/appointments	Any Auth	Book appointment
PATCH	/api/appointments/:id	DOCTOR/ADMIN	Update status/reschedule
GET	/api/audit-logs	ADMIN only	Paginated audit trail
GET	/api/ai/symptom-check	DOCTOR	AI symptom analysis (Ollama)
GET	/api/reports/patient/:id	DOCTOR/ADMIN	Generate PDF report

06. Security Engineering

OWASP Top 10 Mitigation Plan

OWASP Risk	Mitigation	Implementation
A01 Broken Access Control	RBAC + tRPC middleware	Role checked on every procedure before DB query
A02 Cryptographic Failures	AES-256-GCM + TLS 1.3	Records encrypted at rest; HTTPS enforced
A03 Injection	Prisma parameterized queries	Zero raw SQL; Zod input validation
A04 Insecure Design	Threat modeling per feature	Auth flows reviewed pre-implementation
A05 Security Misconfiguration	Env var audit + Helmet.js	No secrets in repo; CSP headers set
A06 Vulnerable Components	Dependabot + npm audit CI	Auto PRs for CVEs; blocked on critical
A07 Auth Failures	JWT + refresh + rate limit	Redis-backed rate limit (5 fails → lockout)
A08 Data Integrity Failures	Signed JWTs + CSP	Supply chain via package-lock integrity
A09 Logging Failures	Append-only audit_logs table	All PHI access logged; log tampering prevented
A10 SSRF	Allowlist outbound domains	AI/external calls restricted to known hosts

Additional Security Measures

- HIPAA Audit Trail: Every access to patient records logged with userId, IP, userAgent, timestamp, action
- Row-Level Security (RLS) in PostgreSQL — patients can ONLY see their own records even with direct DB access
- Medical record content encrypted with AES-256-GCM before storage — DB breach ≠ data breach
- Refresh token rotation — single use, stored as bcrypt hash in Redis with TTL
- CSRF protection via SameSite=Strict cookies + custom header verification
- Content Security Policy (CSP) with strict-dynamic — XSS mitigated at browser level
- Rate limiting: 10 req/min on auth endpoints; 100 req/min on API (Redis sliding window)
- Automated secret scanning with Gitleaks in CI — blocks commits with leaked credentials
- Security headers: HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
- Input sanitization with DOMPurify on all rich-text clinical note fields

07. UI/UX Design System — Neumorphic

The UI follows the neumorphic design language shown in the reference image: soft shadows, extruded cards, off-white palette, orange accent, and minimal typography. Every component feels physically tactile.

Design Tokens

Token	Value	Usage
--color-bg	#EEF0F5	Page background
--color-surface	#F0F2F7	Card / panel surface
--shadow-dark	#C8CAD4	Bottom-right neumorphic shadow
--shadow-light	#FFFFFF	Top-left neumorphic shadow
--color-accent	#FF6B35	Primary CTAs, active states, charts
--color-accent-2	#E84545	Alerts, critical badges
--color-blue	#4A90D9	Info states, links
--text-dark	#2C2C3E	Headings, primary text
--text-mid	#5A5A7A	Body text, descriptions
--text-light	#9090AA	Labels, placeholders, metadata
--radius-card	12px	Card border radius
--radius-pill	999px	Badges, toggles, buttons
--font-sans	Inter, system-ui, sans-serif	All body/UI text

Neumorphic Component Rules

- Cards: box-shadow: 6px 6px 12px #C8CAD4, -6px -6px 12px #FFFFFF — both shadows always present
- Pressed state: box-shadow inset 4px 4px 8px #C8CAD4, inset -4px -4px 8px #FFFFFF
- Toggles (like the OFF/ON in reference): pill shape, inner shadow when OFF, accent fill when ON
- Donut/pie chart: neumorphic extruded ring, segment lifts on hover (translateY -4px + shadow boost)
- Bottom nav: extruded pill container, active icon gets accent dot + scale(1.1) transform
- Form inputs: inset neumorphic (sunken into surface) — focus ring uses accent color
- All interactive elements: transition: all 150ms ease — snappy but not jarring
- Typography scale: 32px heading → 20px subheading → 14px body → 11px label → 9px caption

Key Screens

Screen	Key Components	Role Access
Dashboard / Home	Balance card (Patient summary card), Stat donut, Notification toggle	All roles

Patient Search	Neumorphic search input, filtered list cards, MRN badge	Doctor, Nurse, Admin
Patient Profile	Card carousel (like CC carousel in ref), vitals mini-charts, record timeline	Doctor, Nurse
Appointment Booking	Calendar picker (neumorphic), time slot grid, doctor selector	Patient, Receptionist
Vitals Dashboard	Stat donut (ref right screen), line charts per vital, period selector	Doctor, Nurse
Medical Records	Timeline view, encrypted badge, PDF download button	Doctor, Patient
Admin Panel	Table with role badges, audit log feed, clinic settings	Admin only
AI Symptom Check	Chat-style interface, confidence bars, differential diagnosis cards	Doctor

08. Feature Roadmap

Phase 1 — Foundation (Weeks 1–4)

Phase	Scope	Duration	Priority
P1.1	Replace Firebase Auth with NextAuth v5 (email+password, JWT, refresh tokens)	1 week	CRITICAL
P1.2	Replace Firestore with PostgreSQL + Prisma (schema migration, seed data)	1 week	CRITICAL
P1.3	Implement RBAC middleware — 5 roles, role-checked tRPC procedures	3 days	CRITICAL
P1.4	Rewrite README — project showcase with screenshots, setup guide, architecture	2 days	CRITICAL
P1.5	Docker Compose setup — PostgreSQL, Redis, MinIO, app in one command	2 days	HIGH
P1.6	OWASP hardening pass — CSP, rate limiting, input validation (Zod)	3 days	HIGH

Phase 2 — Core EHR Features (Weeks 5–10)

Phase	Scope	Duration	Priority
P2.1	Patient registration, MRN generation, demographic management	1 week	HIGH
P2.2	Appointment scheduling — booking, calendar view, SMS/email reminders	1 week	HIGH
P2.3	Medical records CRUD — encrypted content, file attachments, version history	1 week	HIGH
P2.4	Vitals recording — BP, HR, SpO2, temperature, weight — time series charts	3 days	HIGH
P2.5	Prescription management — medication list, dosage, drug interaction warnings	4 days	HIGH
P2.6	PDF report generation — BullMQ async pipeline, patient summary PDF download	3 days	MEDIUM
P2.7	Neumorphic design system — design tokens, component library, dark mode	1 week	HIGH

Phase 3 — Advanced Features (Weeks 11–18)

Phase	Scope	Duration	Priority
P3.1	AI symptom checker — Ollama + llama3 local model, structured output	1 week	HIGH
P3.2	Lab results management — CRUD, reference range alerts, trend charts	1 week	MEDIUM
P3.3	Full-text search — Meilisearch for patients, records, medications	3 days	MEDIUM
P3.4	Audit dashboard (Admin) — paginated log viewer, export CSV, alert on anomaly	4 days	HIGH
P3.5	Patient portal — self-service appointment booking, record download, messaging	1 week	MEDIUM
P3.6	Telemedicine prep — video consultation link generation, consent forms	4 days	MEDIUM
P3.7	Notification center — in-app + email + (optional) SMS via free gateway	3 days	MEDIUM

Phase 4 — Enterprise Features (Weeks 19–26)

Phase	Scope	Duration	Priority
P4.1	Multi-clinic tenancy — admin can manage multiple clinic accounts	1 week	MEDIUM
P4.2	Basic billing module — invoice generation, insurance claim tracking	1 week	LOW
P4.3	Analytics dashboard — admission trends, doctor utilization, revenue charts	1 week	MEDIUM
P4.4	FHIR R4 export — standard health record format for interoperability	1 week	LOW
P4.5	Prometheus + Grafana monitoring — request rate, error rate, DB latency	3 days	MEDIUM
P4.6	Two-factor authentication — TOTP (Authenticator app) for staff accounts	3 days	HIGH
P4.7	Offline mode — Service Worker + local IndexedDB sync for unstable connections	1 week	LOW

09. AI-Powered Features (Free/Open-Source)

All AI features use zero-cost models. Ollama runs locally in Docker. HuggingFace Inference API has a generous free tier for smaller models.

Feature	Model / Tool	Benefit
Clinical Symptom Checker	Ollama + llama3.2 (local)	Doctor enters symptoms → differential diagnosis list
Medical Record Summarizer	Ollama + llama3.2 (local)	Summarize patient history for fast doctor review
Drug Interaction Checker	HuggingFace + RxNorm API (free)	Flag dangerous medication combinations
Discharge Summary Generator	Ollama + llama3.2 (local)	Auto-draft discharge notes from structured record da
ICD-10 Code Suggester	Fine-tuned BERT (HuggingFace)	Suggest billing codes from clinical note text
Anomaly Detection (Vitals)	Python Prophet / Z-score (free)	Alert when vitals deviate from patient baseline
OCR for Lab Scans	Tesseract OCR (free, Docker)	Extract values from uploaded lab report PDFs/image
Appointment No-Show Predictor	scikit-learn (local)	Predict cancellations → enable overbooking strategy

AI Integration Architecture

- Ollama runs in a Docker sidecar container — models downloaded at first run, cached in volume
- All AI calls are async via BullMQ — no blocking user requests on LLM inference
- AI responses never auto-modify medical records — always presented as suggestions requiring doctor approval
- Confidence scores shown on all AI outputs — transparency over black-box behavior
- Fallback: if Ollama unavailable, degrade gracefully to manual entry with a warning banner
- AI audit trail: every AI suggestion logged with model version, prompt hash, and doctor response

10. Performance Optimizations

Optimization	Technique	Expected Impact
Database Indexing	B-tree indexes on patient MRN, appointment date, user email	Query time <10ms for common lookups
Connection Pooling	PgBouncer in Docker + Prisma connection limit	Handle 1000+ concurrent users
Redis Caching	Cache patient profiles (TTL 5min), appointment lists	50-70% reduction in DB reads
Image Optimization	Next.js Image component + WebP conversion	60%+ smaller image payloads
API Response Pagination	Cursor-based pagination (keyset) on all list endpoints	Stable performance at any dataset size
Lazy Loading	Dynamic imports for heavy components (charts, PDF viewer)	50%+ faster initial page load
Edge Functions	Next.js middleware at Edge for auth token validation	Auth checks in <5ms globally
Bundle Analysis	next-bundle-analyzer + tree shaking audit	Eliminate unused dependencies
Database Query Optimization	Prisma `select` fields, avoid N+1 with `include`	Reduce data transfer by 70%
Static Generation	ISR for appointment availability pages	Near-instant load for common pages

11. Testing Strategy

Target: >80% coverage across all layers

Test Type	Tool	What's Tested	Coverage Target
Unit Tests	Vitest	Auth utilities, Zod validators, business logic functions	90%+
Integration Tests	Vitest + Prisma test DB	API route handlers, DB transactions, RBAC enforcement	80%+
E2E Tests	Playwright	Complete user flows: register → login → book → consult	Key flows 100%
Security Tests	OWASP ZAP (automated)	SQL injection, XSS, CSRF, broken auth, IDOR	OWASP Top 10
Performance Tests	k6 (free)	Load test: 500 concurrent users, p95 response <200ms	SLA targets
Accessibility	axe-core + Playwright	WCAG 2.1 AA compliance on all pages	100% no violations
API Contract Tests	Zod + Supertest	Validate all API responses match type definitions	100% endpoints
Snapshot Tests	Vitest snapshots	Neumorphic component renders	UI regression

Test Infrastructure Setup

- Separate test database (PostgreSQL in Docker) — migrations auto-run before test suite
- Test data factories using @faker-js/faker for realistic patient/doctor data generation
- Prisma test isolation: each integration test runs in a transaction, rolled back after
- Playwright tests run in GitHub Actions headless Chrome — screenshots on failure
- Coverage report uploaded to Codecov (free for open source) — badge on README
- Pre-commit hooks (Husky + lint-staged): run affected unit tests before every commit

12. DevOps & CI/CD Pipeline

GitHub Actions — 100% Free Tier

Pipeline	Trigger	Jobs
ci.yml — Quality Gate	Every push + PR	lint → type-check → unit tests → integration tests → coverage report
e2e.yml — E2E Tests	PR to main + nightly	Build Docker stack → Playwright tests → screenshot artifacts
security.yml — Security Scan	Every PR	npm audit → Gitleaks secrets scan → OWASP ZAP API scan
deploy-staging.yml	Merge to develop	Build Docker image → push to GHCR → deploy to staging (free Railwa
deploy-prod.yml	Release tag (v*.*.*)	Build → tag image → deploy to prod → smoke tests → notify
dependabot.yml — Deps	Weekly auto	Auto PRs for npm/Docker image updates with security labels
docs.yml — API Docs	Merge to main	Generate Swagger from tRPC schema → publish to GitHub Pages

Docker Compose — One-Command Setup

```
# Start entire CHR-System stack:
git clone https://github.com/Madhavan-dev18/chr-system
cd chr-system
cp .env.example .env
docker compose up -d
# → App: http://localhost:3000
# → MinIO: http://localhost:9001
# → Grafana: http://localhost:3001
```

Free Hosting Options (Zero Cost)

Service	What to Host	Free Limit
Vercel	Next.js app (frontend + API routes)	Hobby: unlimited personal projects
Railway	PostgreSQL + Redis sidecars	$5 free credit/month — enough for demos
GitHub Container Registry	Docker images	Free for public repos
GitHub Pages	API docs (Swagger UI)	Unlimited static hosting
Supabase (free tier)	PostgreSQL alternative	2 free projects, 500MB storage
Upstash	Redis (serverless)	10,000 commands/day free
Cloudflare Pages	Static assets / CDN	Unlimited free tier

13. Documentation Standards

Document	Location	Contents
README.md	Root	Hero banner, live demo link, tech stack badges, 1-command setup, scre
ARCHITECTURE.md	/docs/	C4 system diagram, data flow, tech decisions with rationale
API.md + Swagger UI	/docs/api/	Auto-generated from tRPC schema; every endpoint documented
SECURITY.md	Root	Vulnerability reporting process, security architecture overview
CONTRIBUTING.md	Root	Branch strategy, commit convention (Conventional Commits), PR templ
CHANGELOG.md	Root	Semantic versioning; auto-generated from conventional commits
ADRs (Architecture Decision Records)	/docs/adr/	Record of every major tech decision with context + alternatives
DEPLOYMENT.md	/docs/	Docker, Railway, Vercel, environment variables, SSL setup
HIPAA_COMPLIANCE.md	/docs/	Audit trail design, encryption approach, access control matrix
Storybook	Internal	All neumorphic UI components documented with props and states

14. Priority Matrix

Impact vs Effort Grid

Priority	Item	Impact	Effort	Sprint
CRITICAL	Fix README — showcase real project	Recruiter visibility ↑↑↑	2 days	Week 1
CRITICAL	Replace Firebase with NextAuth + PostgreSQL	Production-ready, no vendor lock-in	2 weeks	Week 1–2
CRITICAL	Implement RBAC (5 roles)	Security, real-world architecture	1 week	Week 3
CRITICAL	OWASP Top 10 hardening	Security credibility	1 week	Week 3–4
HIGH	Docker Compose one-command setup	DX, onboarding	2 days	Week 4
HIGH	Write test suite >80% coverage	Quality, CI gate	2 weeks	Week 5–6
HIGH	Neumorphic UI design system	Visual differentiation	1 week	Week 7
HIGH	Appointment scheduling module	Core EHR feature	1 week	Week 8
HIGH	Audit log dashboard (HIPAA)	Enterprise credibility	4 days	Week 9
HIGH	GitHub Actions CI/CD pipeline	Professional workflow	2 days	Week 4
MEDIUM	AI symptom checker (Ollama)	WOW factor	1 week	Week 11
MEDIUM	PDF report generation	Practical feature	3 days	Week 10
MEDIUM	Meilisearch integration	UX, scalability	3 days	Week 12
MEDIUM	Analytics dashboard	Portfolio depth	1 week	Week 15
LOW	FHIR R4 export	Interoperability	1 week	Week 22
LOW	Offline mode / PWA	Edge case	1 week	Week 24

15. Step-by-Step Implementation Plan

Week	Focus	Deliverables	Success Metric
1	Audit + README	New README with screenshots, live demo setup	GitHub stars ↑, readme renders well
2	Auth Migration	NextAuth v5 with JWT, login/register pages	Login flow working, tokens validated
3	Database Migration	PostgreSQL schema, Prisma migrations, seed data	All tables created, seeds run
4	RBAC + Security	5-role middleware, OWASP hardening, Docker Compose	Role tests pass, ZAP scan clean
5–6	Test Suite	Vitest unit+integration, Playwright E2E, >80% coverage	Coverage badge on README
7	UI Design System	Neumorphic component library, design tokens, Storybook	Components match reference UI
8	Patient Module	Patient CRUD, MRN generation, search	Demo patient flow working
9	Appointments	Booking system, calendar, email reminders	End-to-end booking flow
10	Medical Records	Encrypted records, file upload to MinIO, PDF gen	Record creation + download
11	Vitals + Labs	Time-series vitals, chart dashboard, lab results	Vitals chart renders live data
12	AI Features	Ollama Docker sidecar, symptom checker, summarizer	AI response in <10s
13–14	Search + Notifications	Meilisearch, email notifications, in-app alerts	Patient search <100ms
15–16	Admin + Audit	Admin panel, audit log viewer, anomaly alerts	HIPAA audit trail complete
17–18	CI/CD + Monitoring	GitHub Actions full pipeline, Grafana dashboard	PRs gated on quality
19–26	Enterprise Features	Multi-clinic, billing, FHIR, analytics	Production-grade feature set

16. Final Vision

When complete, CHR-System will be the most comprehensive free, open-source Clinical Health Records platform on GitHub — a showcase that demonstrates every skill a senior full-stack engineer is expected to have.

What Recruiters Will See

- Production-grade Next.js 15 architecture with App Router, tRPC, and TypeScript
- Real security engineering: RBAC, JWT rotation, AES-256-GCM, OWASP hardening
- HIPAA-aligned audit trail — signals enterprise healthcare domain knowledge
- AI integration (Ollama/HuggingFace) — shows you can embed ML into real systems
- Complete DevOps: Docker, GitHub Actions CI/CD, monitoring, error tracking
- High test coverage with Vitest + Playwright — signals professional quality mindset
- Neumorphic design system — demonstrates UI/UX taste beyond Bootstrap defaults
- Open-source community signals: clear docs, CONTRIBUTING.md, clean git history
- 53 commits already — demonstrates sustained effort, not a weekend project
- Multi-domain depth: healthcare domain, security, AI, DevOps, UI — rare combination

CHR-System after this PRD is not just another CRUD app. It is a demonstrated ability to architect, secure, test, deploy, and document a system that handles real sensitive data at scale — the exact bar that separates junior from mid-to-senior engineers. It is the portfolio project that answers every recruiter question before it's asked.

Madhavan Shivakumar · github.com/Madhavan-dev18 · madhavan-shivakumar-dev.vercel.app · June 2026